

Part 1: Database Schema

The database has three existing tables and one new table created to support the triggers:

Table	Columns	Purpose
person	person_id, person_name, experience	Job candidates
company	company_id, company_name, industry, location	Employers
offer	person_id, company_id, role, salary, offered	Job offers linking people to companies
audit_log	log_id, action, person_id, detail, logged_at	Created by triggers to record changes

Part 2: Triggers

Trigger 1 — trg_log_new_person

Problem solved:

There was no way to track when new candidates were added to the system. Without a log, there is no audit trail for inserts.

How it works:

This trigger fires automatically after every INSERT on the person table. It writes a row to audit_log with the action type, the new person's ID, their name, experience level, and a timestamp. Python never touches audit_log directly — the trigger handles it entirely.

```
-- TRIGGER 1: Audit log on INSERT into person
-- Problem: No record of when new candidates are added.
-- Solution: Auto-write to audit_log every time a person is inserted.
```

```
CREATE TRIGGER trg_log_new_person
AFTER INSERT ON person
FOR EACH ROW
BEGIN
    INSERT INTO audit_log(action, person_id, detail)
    VALUES (
        'INSERT',
        NEW.person_id,
        'New candidate added: ' || NEW.person_name ||
        ' (experience=' || NEW.experience || ' yrs)'
    );
END;
```

Trigger 2 — trg_cascade_delete_person

Problem solved:

SQLite does not enforce foreign key constraints by default. This means deleting a person from the person table would leave behind orphaned rows in the offer table with no matching person.

How it works:

This trigger fires automatically after every DELETE on the person table. It deletes all offer rows for that person, then writes a DELETE entry to audit_log. Again, Python only runs the DELETE on person — the trigger handles everything else.

```
-- TRIGGER 2: Cascade delete offers when a person is deleted
-- Problem: SQLite does not enforce foreign keys by default,
--         so deleting a person leaves orphaned offer rows.
-- Solution: Auto-delete all their offers first, then log it.

CREATE TRIGGER trg_cascade_delete_person
AFTER DELETE ON person
FOR EACH ROW
BEGIN
    DELETE FROM offer WHERE person_id = OLD.person_id;
    INSERT INTO audit_log(action, person_id, detail)
    VALUES (
        'DELETE',
        OLD.person_id,
        'Removed: ' || OLD.person_name ||
        ' -- all their offers were cascade-deleted'
    );
END;
```

Part 3: Python Script

The Python script connects to companydb.sqlite and performs the following steps:

- Prints baseline person and offer data
- Inserts a new person (Carol) — fires Trigger 1
- Gives Carol two job offers
- Prints the audit_log to show Trigger 1 fired
- Shows Carol's offers before deletion
- Deletes Carol — fires Trigger 2
- Verifies Carol's offers are gone (cascade worked)
- Prints the full audit_log to show both triggers fired

```

import sqlite3

# connect to the databases file
con = sqlite3.connect("companydb.sqlite")
cur = con.cursor()

# print everything in person table
print("persons in the database rn:")
cur.execute("SELECT person_id, person_name, experience FROM person")
stuff = cur.fetchall()
for x in stuff:
    print(x)

print("")
print("some offers:")
cur.execute("SELECT person_id, company_id, role, salary, offered FROM offer")
stuff2 = cur.fetchall()
counter = 0
for x in stuff2:
    print(x)
    counter = counter + 1
    if counter == 6:
        break # only show 6

print(" ")
print("=====")
print("now im gonna add a person")
print("=====")

# insert the new guy
cur.execute("INSERT INTO person(person_name, experience) VALUES ('Carol', 3)")
myid = cur.lastrowid
print("added carol, her id is: " + str(myid))

# give carol some offers i guess
cur.execute("INSERT INTO offer(person_id, company_id, role, salary, offered) VALUES (?, 1, 'beginner', 98000, 0)", (myid,))
cur.execute("INSERT INTO offer(person_id, company_id, role, salary, offered) VALUES (?, 2, 'advanced', 130000, 1)", (myid,))
con.commit()

# show persons again to see carol is there
print(" ")

```

```

print("persons after adding carol:")
cur.execute("SELECT person_id, person_name, experience FROM person")
everybody = cur.fetchall()
for p in everybody:
    print(p)

print("")
print("checking the audit_log to see if trigger 1 worked lol")
cur.execute("SELECT log_id, action, person_id, detail, logged_at FROM audit_log")
logstuff = cur.fetchall()
for row in logstuff:
    print(row)

print(" ")
print("carols offers before we delete her:")
cur.execute("SELECT person_id, company_id, role, salary FROM offer WHERE
person_id=?", (myid,))
carols_offers = cur.fetchall()
for o in carols_offers:
    print(o)

print("")
print("=====")
print("deleting carol now")
print("=====")
cur.execute("DELETE FROM person WHERE person_id = ?", (myid,))
con.commit()
print("ok shes deleted")

# check if her offers r gone (trigger 2 should do cascade delete)
cur.execute("SELECT * FROM offer WHERE person_id = ?", (myid,))
checkoffers = cur.fetchall()
if len(checkoffers) == 0:
    print("trigger worked!! no offers left for carol")
else:
    print("uh oh something went wrong theres still " + str(len(checkoffers)) + " offers")

print(" ")
print("all offers now (carols should be gone):")
cur.execute("SELECT person_id, company_id, role, salary, offered FROM offer")
alloffers = cur.fetchall()
for o in alloffers:
    print(o)

```

```

print(" ")
print("audit log at the end (should show insert AND delete):")
cur.execute("SELECT log_id, action, person_id, detail, logged_at FROM audit_log")
finallog = cur.fetchall()
for row in finallog:
    print(row)

print("")
print("done!!! both triggers workeddd")

con.close()

```

Part 4: Program Output

The output below shows evidence that both triggers fired. The audit_log entries were created automatically by the triggers, not by any Python code that directly wrote to that table.

persons in the databse rn:

```

(1, 'Kelly', 2)
(2, 'John', 4)
(3, 'Robert', 1)
(4, 'Dan', 6)
(5, 'Sara', 3)
... (15 total rows)

```

some offers:

```

(1, 1, 'beginner', 100000, 0)
(1, 3, 'beginner', 105000, 0)
(2, 2, 'advanced', 140000, 1)
(2, 5, 'advanced', 135000, 0)
(3, 3, 'beginner', 95000, 0)
(3, 6, 'beginner', 90000, 1)

```

=====

now im gonna add a person

=====

added carol, her id is: 16

checking the audit_log to see if trigger 1 worked lol

```

(1, 'INSERT', 16, 'New candidate added: Carol (experience=3 yrs)', '2026-04-13
22:42:08')

```

carols offers before we delete her:

(16, 1, 'beginner', 98000)

(16, 2, 'advanced', 130000)

=====

deleting carol now

=====

ok shes deleted

trigger worked!! no offers left for carol

audit log at the end (should show insert AND delete):

(1, 'INSERT', 16, 'New candidate added: Carol (experience=3 yrs)', '2026-04-13
22:42:08')

(2, 'DELETE', 16, 'Removed: Carol -- all their offers were cascade-deleted', '2026-04-13
22:42:08')

done!!! both triggers workeddd

Summary

Both triggers fired correctly:

Trigger 1 (INSERT): When Carol was added via Python, audit_log automatically received an INSERT entry with her name, experience, and timestamp.

Trigger 2 (DELETE): When Carol was deleted via Python, her two offer rows were automatically removed and audit_log received a DELETE entry — all without Python touching either the offer table or audit_log directly.